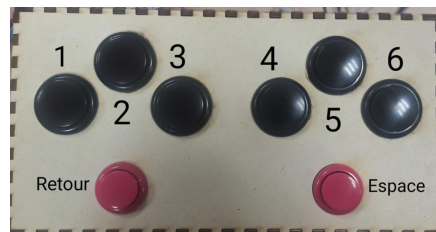


Guide de compréhension du code STM32

Clavier braille autonome pour BrailleRAP



Projet : Clavier braille autonome pour BrailleRAP

Réalisé par : Rhazouane Douaa et Oumayma Adebji

Formation : Master 1 EEEA – Systèmes Embarqués

Établissement : ISTIC – Université de Rennes

Année universitaire : 2025 – 2026

Table des matières

1	Introduction	3
2	Installation des logiciels	3
2.1	Installation de STM32CubeIDE	3
3	Création d'un projet STM32CubeIDE	4
4	Configuration du fichier .ioc	5
4.1	Réinitialisation de la configuration	5
4.2	Configuration de l'USART1	5
4.3	Configuration des boutons en entrée GPIO avec Pull-up	6
4.4	Génération automatique du code	8
4.5	Connexion entre la STM32 et la carte MKS Gen L	9
5	Explication de la boucle principale	9
6	Lecture des boutons avec Pull-up	10
7	Reconnaissance des lettres	10
8	Pourquoi utiliser des fonctions pour chaque lettre ?	11
9	Structure générale du code	11
10	Les bibliothèques	12
11	Les variables globales	12
12	Les prototypes de fonctions	12
13	La fonction main	13
14	La boucle while(1)	13
15	Lecture des boutons	13
16	Pourquoi lire les six boutons ?	14
17	Reconnaissance des combinaisons	14
18	Stabilisation de l'appui	14
19	Attente du relâchement des boutons	15
20	La fonction sendGcode	15
21	La fonction movePoint	15
22	La fonction hitMagnet	15

23 Exemple : impression de la lettre B	16
24 Gestion de l'espace	16
25 Retour à la ligne	17
26 Éjection de la feuille	17
27 Comment ajouter une nouvelle lettre ou un symbole ?	17
28 Solution provisoire au problème de l'axe Y	18
29 Conclusion	19

1 Introduction

Ce document explique le fonctionnement du code développé pour la carte STM32 utilisée dans le projet de clavier braille autonome pour la BrailleRAP.

L'objectif est de permettre à une personne, même débutante en électronique ou en programmation embarquée, de comprendre le rôle de chaque partie du programme.

Le code permet à la carte STM32 de lire les boutons du clavier braille, d'identifier les combinaisons correspondant aux caractères, puis d'envoyer des commandes G-code à la carte MKS Gen L afin de piloter la BrailleRAP.

2 Installation des logiciels

2.1 Installation de STM32CubeIDE

Le développement du programme STM32 a été réalisé avec le logiciel **STM32CubeIDE**, l'environnement de développement officiel de STMicroelectronics. Ce logiciel permet de créer, compiler, déboguer et téléverser le programme sur la carte STM32.

Au moment de la rédaction de ce guide, la version recommandée est la **1.17.0**, car elle est stable et compatible avec le projet présenté.

Le logiciel peut être téléchargé depuis le site officiel de STMicroelectronics :

<https://www.st.com/en/development-tools/stm32cubeide.html>

Les étapes d'installation sont les suivantes :

1. Accéder au site officiel de STMicroelectronics.
2. Télécharger **STM32CubeIDE** (version recommandée : **1.17.0**).
3. Installer le logiciel en suivant l'assistant d'installation.
4. Lancer STM32CubeIDE.
5. Créer un nouveau projet ou ouvrir un projet existant.

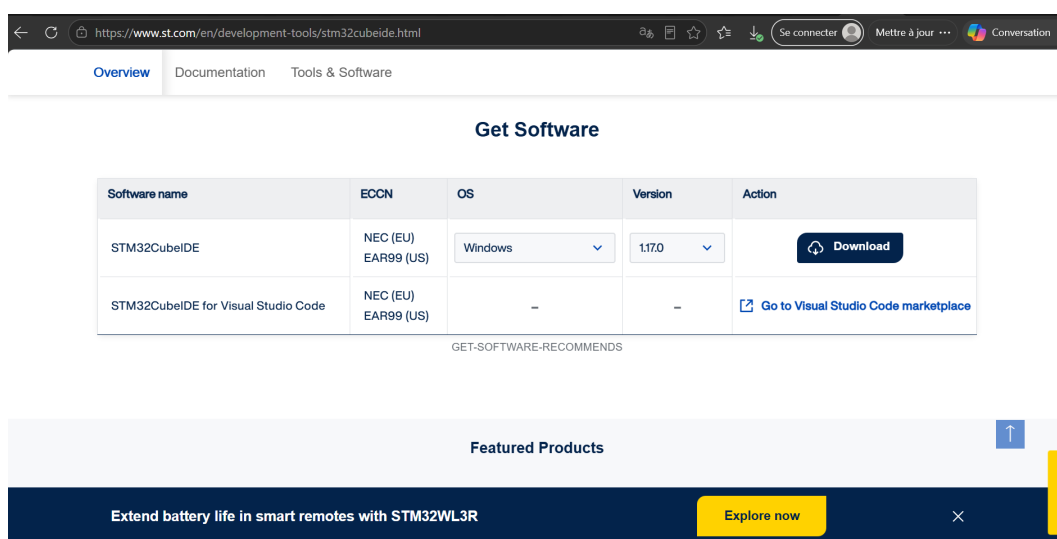


FIGURE 1 – Page officielle de téléchargement de STM32CubeIDE

3 Création d'un projet STM32CubeIDE

Après avoir installé STM32CubeIDE, lancer le logiciel. Lors du premier démarrage, il est demandé de choisir un dossier de travail (*Workspace*) dans lequel seront enregistrés les différents projets.

Une fois le logiciel ouvert, créer un nouveau projet en suivant le chemin :

File → New → STM32 Project

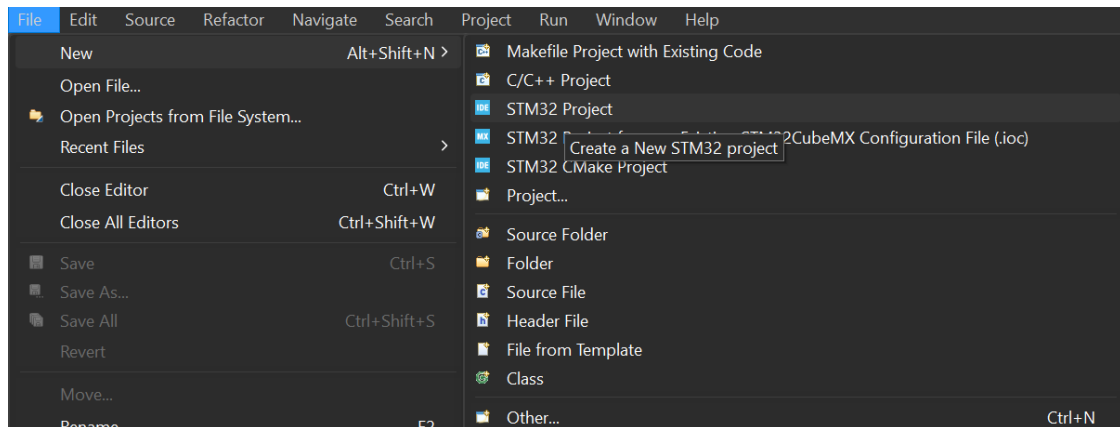


FIGURE 2 – Création d'un nouveau projet STM32

Une fenêtre de sélection de la carte apparaît. Dans notre projet, nous utilisons une carte **NUCLEO-F303RE**. Il suffit donc :

1. Cliquer sur l'onglet **Board Selector**.
2. Rechercher **NUCLEO-F303RE**.
3. Sélectionner la carte.
4. Cliquer sur **Next**.

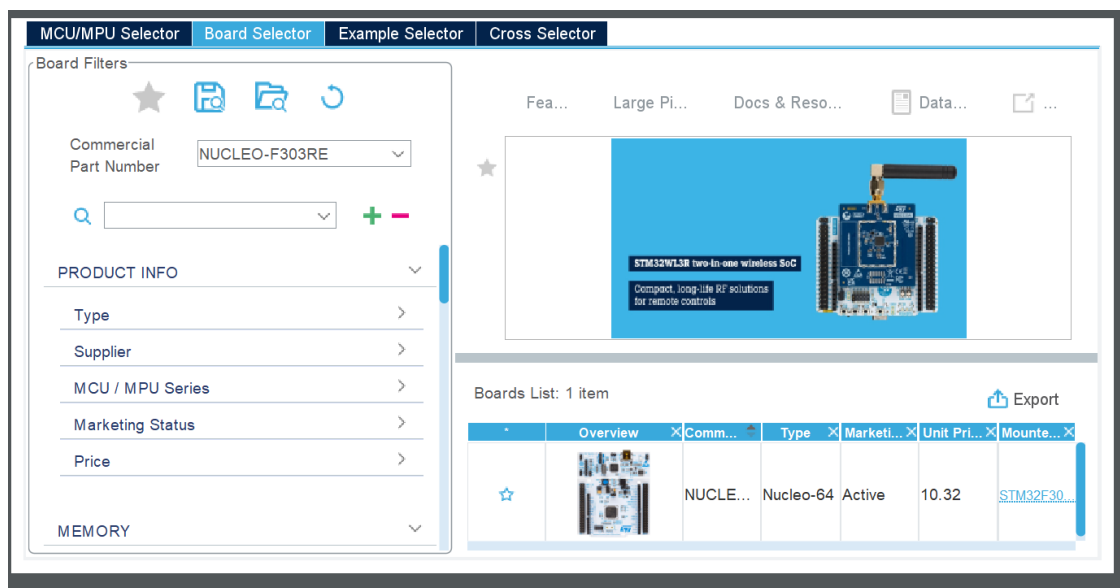


FIGURE 3 – Sélection de la carte STM32

Ensuite, choisir un nom pour le projet, par exemple :

BrailleRAP_Keyboard

Dans la fenêtre de configuration, conserver le langage **C**, choisir **Executable** comme type de binaire et sélectionner **Empty** dans le champ **Targeted Project Type**. Enfin, cliquer sur **Finish**.

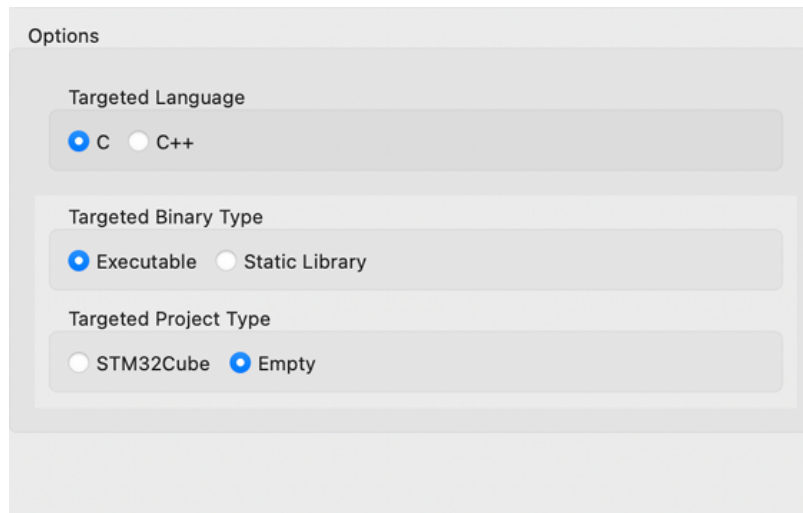


FIGURE 4 – Configuration du projet STM32CubeIDE

4 Configuration du fichier .ioc

Après la création du projet, STM32CubeIDE ouvre automatiquement le fichier **.ioc**. Ce fichier permet de configurer tous les périphériques de la carte STM32 (GPIO, UART, Timers, etc.) sans écrire de code manuellement.

4.1 Réinitialisation de la configuration

Avant de commencer la configuration, il est conseillé de repartir d'une configuration propre.

Pour cela :

1. Ouvrir le fichier **.ioc**.
2. Cliquer sur **Clear Pinout**.
3. Confirmer la réinitialisation des broches.

Cette étape permet de supprimer toute configuration précédente et d'éviter les conflits entre périphériques.

4.2 Configuration de l'USART1

La communication entre la carte STM32 et la carte MKS Gen L est réalisée grâce au périphérique **USART1**.

Pour le configurer :

1. Dans le menu de gauche, ouvrir **Connectivity**.

2. Sélectionner **USART1**.
3. Dans le champ **Mode**, choisir **Asynchronous**.
4. Vérifier que **Hardware Flow Control** est désactivé.
5. Dans l'onglet **Parameter Settings**, régler les paramètres suivants :

TABLE 1 – Configuration de l'USART1

Paramètre	Valeur
Mode	Asynchronous
Baud Rate	250000 Bits/s
Word Length	8 Bits
Parity	None
Stop Bits	1
Hardware Flow Control	Disable

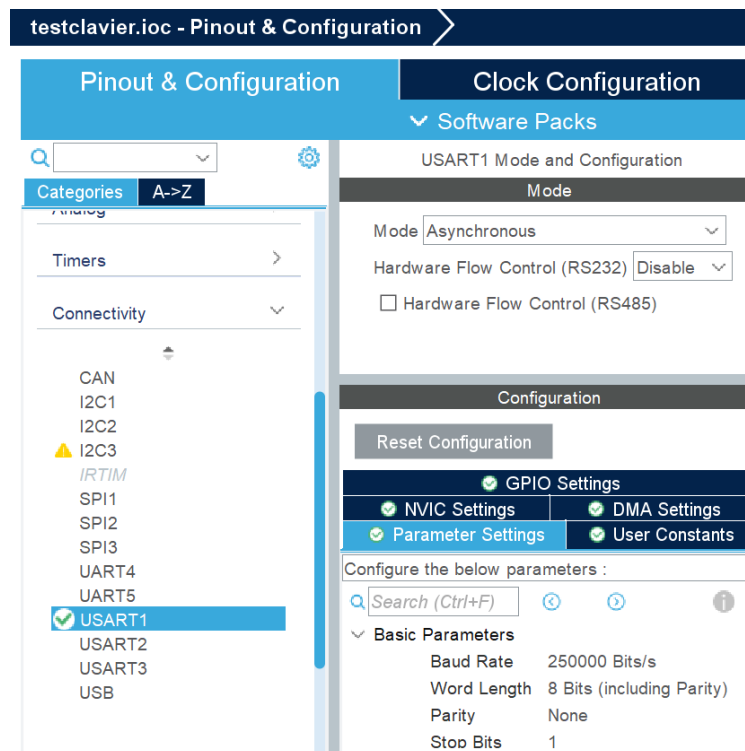


FIGURE 5 – Configuration de l'USART1 dans STM32CubeIDE

Ces paramètres sont indispensables pour assurer une communication correcte avec le firmware Marlin exécuté sur la carte MKS Gen L.

4.3 Configuration des boutons en entrée GPIO avec Pull-up

Les boutons du clavier braille doivent être configurés comme des entrées numériques. Chaque bouton est relié d'un côté à une broche de la STM32 et de l'autre côté à la masse. Pour que la lecture soit stable, les broches sont configurées en mode **GPIO_Input** avec une résistance interne **Pull-up**.

Pour configurer une broche :

1. Cliquer sur la broche souhaitée dans le schéma de la carte.
2. Choisir **GPIO_Input**.
3. Aller dans **System Core** puis **GPIO**.
4. Vérifier que le mode est **Input mode**.
5. Régler **GPIO Pull-up/Pull-down** sur **Pull-up**.

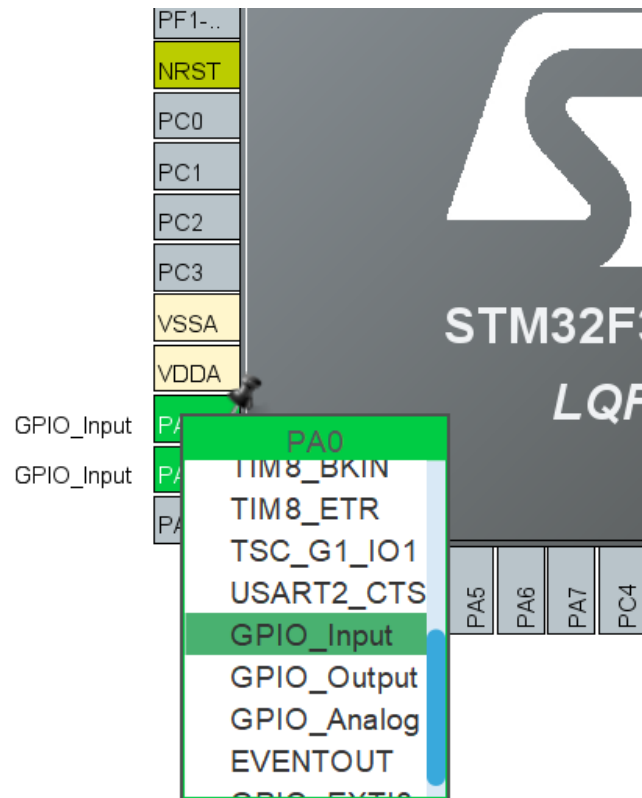


FIGURE 6 – Sélection du mode GPIO_Input pour une broche

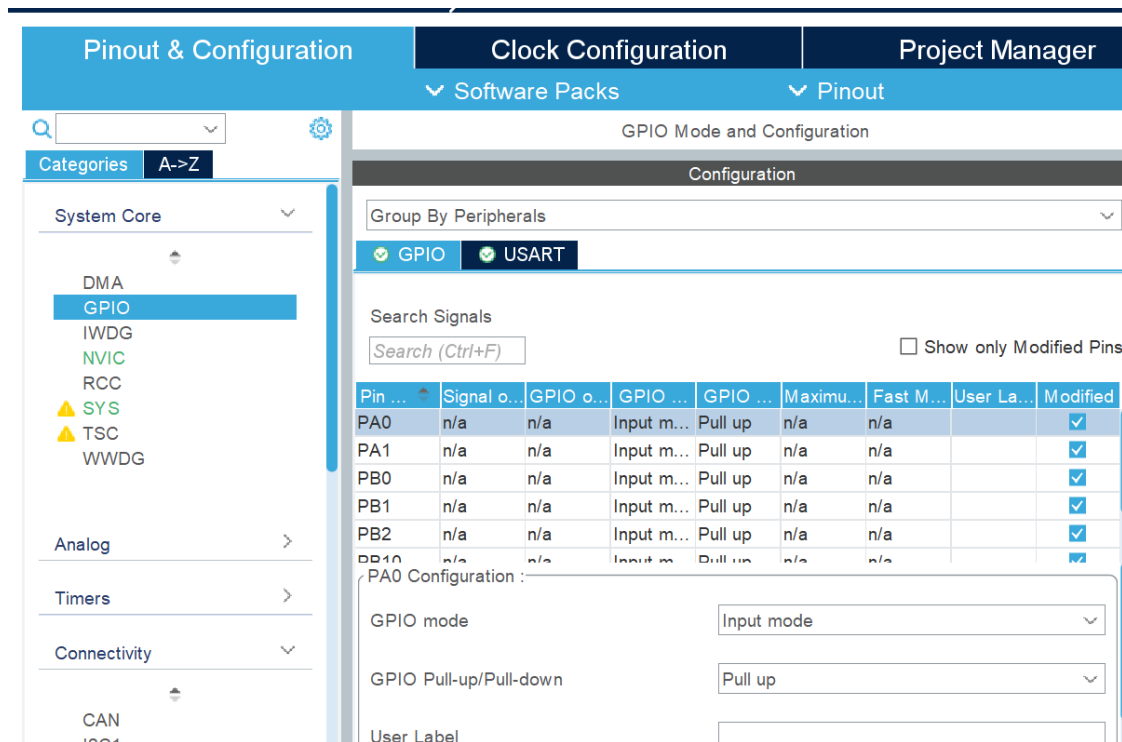


FIGURE 7 – Configuration des entrées GPIO en mode Pull-up

Les broches utilisées pour les boutons sont les suivantes :

TABLE 2 – Broches configurées pour les boutons du clavier

Broche STM32	Bouton	Configuration
PA0	Point braille 1	GPIO Input + Pull-up
PA1	Point braille 2	GPIO Input + Pull-up
PB0	Point braille 3	GPIO Input + Pull-up
PB1	Point braille 4	GPIO Input + Pull-up
PB2	Point braille 5	GPIO Input + Pull-up
PB10	Point braille 6	GPIO Input + Pull-up
PB11	Retour à la ligne	GPIO Input + Pull-up
PB12	Espace	GPIO Input + Pull-up

Avec cette configuration, lorsqu'un bouton n'est pas appuyé, la broche est à l'état logique haut. Lorsqu'un bouton est appuyé, il relie la broche à la masse, ce qui donne un état logique bas. C'est pour cela que dans le programme, un bouton appuyé est détecté avec l'état **GPIO_PIN_RESET**.

4.4 Génération automatique du code

Une fois la configuration du projet terminée (GPIO, USART, etc.), il est nécessaire de générer le code source correspondant.

Deux méthodes sont possibles :

- Cliquer sur l'icône **Generate Code** située dans la barre d'outils de STM32CubeIDE ;
- Ou utiliser le raccourci clavier **Ctrl + S** (ou **Cmd + S** sous macOS), qui sauvegarde le fichier **.ioc** et lance automatiquement la génération du code.

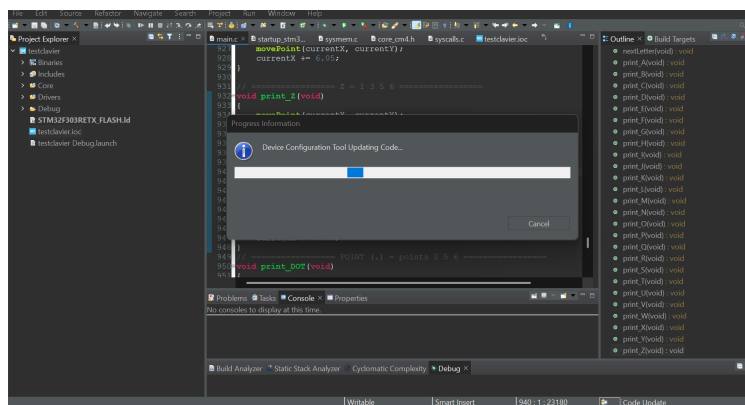


FIGURE 8 – Génération automatique du code dans STM32CubeIDE

Après cette étape, STM32CubeIDE crée automatiquement l'architecture du projet ainsi que les principaux fichiers, notamment :

- Core/Src/main.c
- Core/Inc/main.h
- stm32f3xx_hal_msp.c
- stm32f3xx_it.c
- les fichiers de démarrage et de configuration du projet.

Ces fichiers constituent la base du programme. Le développement de l'application se fera principalement dans le fichier `main.c`, tandis que les autres fichiers sont générés automatiquement par STM32CubeIDE et assurent l'initialisation et la gestion des périphériques de la carte.

4.5 Connexion entre la STM32 et la carte MKS Gen L

La communication entre la carte STM32 et la carte MKS Gen L est réalisée par une liaison série UART. Dans notre projet, la broche **TX** de la STM32 (**PA9 - USART1_TX**) est reliée à la broche **RX0** de la carte MKS Gen L afin de transmettre les commandes G-code.

STM32	MKS Gen L
TX (PA9 - USART1_TX)	RX0
GND	GND

TABLE 3 – Connexion UART entre la STM32 et la carte MKS Gen L

Ainsi, la STM32 envoie les commandes G-code à la carte MKS Gen L par l'intermédiaire de sa sortie TX. Les deux cartes doivent aussi partager une masse commune, c'est-à-dire GND avec GND.

5 Explication de la boucle principale

La partie la plus importante du programme se trouve dans la boucle `while(1)`. Cette boucle permet à la STM32 de surveiller les boutons en permanence.

```
1 while (1)
2 {
```

```

3   int b1 = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET)
    ;
4   int b2 = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_RESET)
    ;
5   int b3 = (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_0) == GPIO_PIN_RESET)
    ;
6   int b4 = (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_1) == GPIO_PIN_RESET)
    ;
7   int b5 = (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_2) == GPIO_PIN_RESET)
    ;
8   int b6 = (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_10) == GPIO_PIN_RESET)
    );
9 }

```

Chaque variable correspond à un bouton du clavier braille :

TABLE 4 – Correspondance entre les boutons et les points braille

Variable	Point braille	Broche STM32
b1	Point 1	PA0
b2	Point 2	PA1
b3	Point 3	PB0
b4	Point 4	PB1
b5	Point 5	PB2
b6	Point 6	PB10

Il est nécessaire de lire les six boutons en même temps car une lettre braille est formée par une combinaison de plusieurs points.

6 Lecture des boutons avec Pull-up

Les boutons sont configurés en entrée avec résistance interne Pull-up. Cela signifie que :

- si le bouton n'est pas appuyé, la broche est à l'état haut ;
- si le bouton est appuyé, la broche est reliée à la masse et passe à l'état bas.

C'est pour cette raison que le programme considère qu'un bouton est appuyé lorsque la lecture vaut `GPIO_PIN_RESET`.

```

1  int b1 = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET);

```

Cette ligne signifie :

- lire l'état de la broche PA0 ;
- vérifier si elle est à l'état bas ;
- si oui, le bouton 1 est considéré comme appuyé.

7 Reconnaissance des lettres

Après la lecture des boutons, le programme compare la combinaison détectée avec les combinaisons braille connues.

```

1  // A = point 1
2  if (b1 && !b2 && !b3 && !b4 && !b5 && !b6)

```

```

3   print_A();
4
5   // B = points 1 et 2
6   else if (b1 && b2 && !b3 && !b4 && !b5 && !b6)
7       print_B();

```

Le symbole && signifie « et ». Le symbole ! signifie « non ».

Par exemple :

```

1   b1 && !b2

```

signifie que le bouton 1 est appuyé et que le bouton 2 n'est pas appuyé.

8 Pourquoi utiliser des fonctions pour chaque lettre ?

Chaque lettre possède sa propre fonction, par exemple :

```

1   void print_A(void);
2   void print_B(void);
3   void print_C(void);

```

Cette organisation permet de rendre le code plus clair. Chaque fonction contient uniquement les déplacements nécessaires pour imprimer la lettre correspondante.

Par exemple, la lettre B correspond aux points 1 et 2 :

```

1   void print_B(void)
2   {
3       movePoint(currentX, currentY);
4       hitMagnet();
5
6       movePoint(currentX, currentY + 5.00);
7       hitMagnet();
8
9       movePoint(currentX, currentY);
10      currentX += 6.05;
11  }

```

La fonction réalise les étapes suivantes :

1. déplacement vers le point 1 ;
2. activation de l'électroaimant ;
3. déplacement vers le point 2 ;
4. activation de l'électroaimant ;
5. retour à la position de référence ;
6. passage à la cellule suivante.

9 Structure générale du code

Le programme est organisé en plusieurs parties :

- les inclusions de bibliothèques
- les variables globales

- les prototypes de fonctions
- la fonction principale `main()`
- la boucle infinie `while(1)`
- les fonctions de communication
- les fonctions d'impression des lettres

10 Les bibliothèques

Au début du programme, on trouve des lignes de type :

```
1 #include "main.h"
2 #include <string.h>
3 #include <stdio.h>
```

Ces bibliothèques permettent d'utiliser des fonctions déjà existantes.

Par exemple :

- `main.h` contient les définitions liées au projet STM32;
- `string.h` permet d'utiliser certaines fonctions sur les chaînes de caractères;
- `stdio.h` permet d'utiliser `sprintf()` pour créer des commandes G-code.

11 Les variables globales

Dans le programme, deux variables sont utilisées pour mémoriser la position actuelle :

```
1 float currentX = 3.00;
2 float currentY = 10.50;
```

Ces variables permettent de savoir où se trouve la prochaine cellule braille à imprimer.

- `currentX` représente la position horizontale;
- `currentY` représente la position verticale.

Après chaque lettre imprimée, `currentX` est augmenté afin de passer à la cellule suivante.

12 Les prototypes de fonctions

Avant la fonction `main()`, on déclare plusieurs fonctions :

```
1 void sendGcode(char *cmd);
2 void movePoint(float x, float y);
3 void hitMagnet(void);
4 void print_A(void);
5 void print_B(void);
```

Ces lignes sont appelées des **prototypes**.

Elles indiquent au compilateur que ces fonctions existent et qu'elles seront définies plus loin dans le programme.

Cela permet d'appeler une fonction dans le code même si sa définition complète se trouve plus bas.

13 La fonction main

La fonction `main()` est le point de départ du programme.

```
1 int main(void)
2 {
3     HAL_Init();
4     SystemClock_Config();
5     MX_GPIO_Init();
6     MX_USART1_UART_Init();
7
8     sendGcode("G28_X");
9     sendGcode("G28_Y");
10    sendGcode("G1_F4000");
11
12    while (1)
13    {
14        // programme principal
15    }
16 }
```

Au démarrage :

1. la STM32 est initialisée;
2. les GPIO sont configurés;
3. l'UART est démarré;
4. la machine fait le référencement des axes X et Y;
5. le programme entre dans la boucle infinie.

14 La boucle while(1)

La boucle `while(1)` permet au programme de fonctionner en continu.

```
1 while (1)
2 {
3     // lecture des boutons
4     // identification de la combinaison
5     // impression de la lettre
6 }
```

La STM32 doit toujours attendre une action de l'utilisateur. C'est pourquoi le programme ne s'arrête jamais.

15 Lecture des boutons

Chaque bouton est lu avec la fonction `HAL_GPIO_ReadPin()`.

```
1 int b1 = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET);
2 int b2 = (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_RESET);
3 int b3 = (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_0) == GPIO_PIN_RESET);
```

Comme les boutons sont configurés en Pull-up :

- bouton relâché : état haut ;
- bouton appuyé : état bas ;
- donc on teste si la broche est égale à `GPIO_PIN_RESET`.

16 Pourquoi lire les six boutons ?

Une cellule braille est composée de six points :

1	4
2	5
3	6

Chaque bouton du clavier correspond à un point braille.

Pour connaître le caractère demandé par l'utilisateur, il faut donc lire les six boutons en même temps.

Par exemple :

- A = point 1 ;
- B = points 1 et 2 ;
- C = points 1 et 4.

17 Reconnaissance des combinaisons

Après la lecture des boutons, le programme utilise des conditions `if` et `else if`.

```

1 // A = point 1
2 if (b1 && !b2 && !b3 && !b4 && !b5 && !b6)
3     print_A();
4
5 // B = points 1 et 2
6 else if (b1 && b2 && !b3 && !b4 && !b5 && !b6)
7     print_B();

```

Chaque condition correspond à une combinaison braille.

Le symbole `&&` signifie "et". Le symbole `!` signifie "non".

Par exemple :

```

1 b1 && !b2

```

signifie :

- le bouton 1 est appuyé ;
- le bouton 2 n'est pas appuyé.

18 Stabilisation de l'appui

Après une première détection, un délai est ajouté :

```

1 HAL_Delay(1000);

```

Ce délai permet de laisser à l'utilisateur le temps d'appuyer sur tous les boutons nécessaires avant que la combinaison soit analysée.

La STM32 relit ensuite les boutons pour vérifier la combinaison complète.

19 Attente du relâchement des boutons

Après l'exécution d'une action, le programme attend que tous les boutons soient relâchés.

```
1 while (HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_0) == GPIO_PIN_RESET ||
2       HAL_GPIO_ReadPin(GPIOA, GPIO_PIN_1) == GPIO_PIN_RESET);
```

Cela évite qu'une même lettre soit imprimée plusieurs fois lorsque l'utilisateur garde les boutons appuyés.

20 La fonction sendGcode

La fonction `sendGcode()` permet d'envoyer une commande à la carte MKS.

```
1 void sendGcode(char *cmd)
2 {
3     HAL_UART_Transmit(&huart1, (uint8_t*)cmd, strlen(cmd),
4                       HAL_MAX_DELAY);
5     HAL_UART_Transmit(&huart1, (uint8_t*)"\r\n", 2, HAL_MAX_DELAY);
6     HAL_Delay(100);
7 }
```

La STM32 n'agit pas directement sur les moteurs. Elle envoie une instruction texte, appelée G-code, à la carte MKS.

21 La fonction movePoint

La fonction `movePoint()` déplace la tête vers une position précise.

```
1 void movePoint(float x, float y)
2 {
3     char cmd[50];
4
5     sprintf(cmd, "G1_X%.2f_Y%.2f_F4000", x, y);
6     sendGcode(cmd);
7
8     HAL_Delay(200);
9     sendGcode("M400");
10 }
```

La fonction construit une commande G-code avec les coordonnées souhaitées, puis l'envoie à la MKS.

M400 permet d'attendre la fin du déplacement avant de continuer.

22 La fonction hitMagnet

La fonction `hitMagnet()` permet de créer un point braille.

```
1 void hitMagnet(void)
2 {
3     sendGcode("M3_S255");
4     HAL_Delay(500);
5 }
```



```

5     sendGcode("M3_S0");
6     HAL_Delay(300);
7 }

```

Son fonctionnement est le suivant :

1. activation de l'électroaimant ;
2. attente pour permettre la frappe ;
3. désactivation de l'électroaimant ;
4. petite pause avant la suite.

23 Exemple : impression de la lettre B

La lettre B correspond aux points 1 et 2.

```

1 void print_B(void)
2 {
3     movePoint(currentX, currentY);
4     hitMagnet();
5
6     movePoint(currentX, currentY + 5.00);
7     hitMagnet();
8
9     movePoint(currentX, currentY);
10    currentX += 6.05;
11 }

```

Explication :

1. déplacement vers le point 1 ;
2. frappe du point 1 ;
3. déplacement vers le point 2 ;
4. frappe du point 2 ;
5. retour à la position de référence ;
6. passage à la cellule suivante.

24 Gestion de l'espace

La fonction `spaceBraille()` avance la position sans imprimer de point.

```

1 void spaceBraille(void)
2 {
3     currentX += 12.10;
4     movePoint(currentX, currentY);
5 }

```

Cela permet de créer un espace entre deux mots.

25 Retour à la ligne

La fonction `newLine()` permet de passer à la ligne suivante.

```
1 void newLine(void)
2 {
3     currentX = 3.00;
4     currentY += 9.90;
5     movePoint(currentX, currentY);
6 }
```

La position horizontale revient au début, et la position verticale augmente pour passer à la ligne suivante.

26 Éjection de la feuille

La fonction `ejectPaper()` permet de faire sortir automatiquement la feuille.

```
1 void ejectPaper(void)
2 {
3     sendGcode("G91");
4     sendGcode("G1_Y270_F3000");
5     sendGcode("M400");
6     sendGcode("G90");
7
8     currentX = 3.00;
9     currentY = 10.50;
10 }
```

Le mode relatif `G91` permet de déplacer l'axe Y par rapport à sa position actuelle. Après l'éjection, le programme revient en mode absolu avec `G90`.

27 Comment ajouter une nouvelle lettre ou un symbole ?

Pour ajouter un nouveau caractère, il faut :

1. connaître sa combinaison braille ;
2. créer une fonction d'impression ;
3. ajouter la condition correspondante dans la boucle principale.

Exemple pour un symbole :

```
1 void print_COMMA(void)
2 {
3     movePoint(currentX, currentY + 5.00);
4     hitMagnet();
5
6     movePoint(currentX, currentY);
7     currentX += 6.05;
8 }
```

Puis dans le `while` :

```

1 else if (!b1 && b2 && !b3 && !b4 && !b5 && !b6)
2     print_COMMA();

```

28 Solution provisoire au problème de l'axe Y

Lors des essais expérimentaux, il a été constaté que le moteur de l'axe **Y** peut se bloquer pendant le fonctionnement de la machine. Ce comportement semble être lié à un problème matériel, probablement au niveau du driver de la carte MKS.

Une solution provisoire a toutefois permis de limiter ce problème. Il suffit de connecter la carte **MKS Gen L** à un ordinateur via son interface USB, puis de lancer le logiciel de pilotage. Ensuite, en utilisant les commandes de déplacement manuel, il est recommandé de faire fonctionner simultanément les axes **X** et **Y**. Cette manipulation permet de renforcer le fonctionnement de l'axe Y et d'éviter son blocage pendant l'impression.

La Figure 9 montre l'interface du logiciel utilisée pour commander manuellement les déplacements des axes.

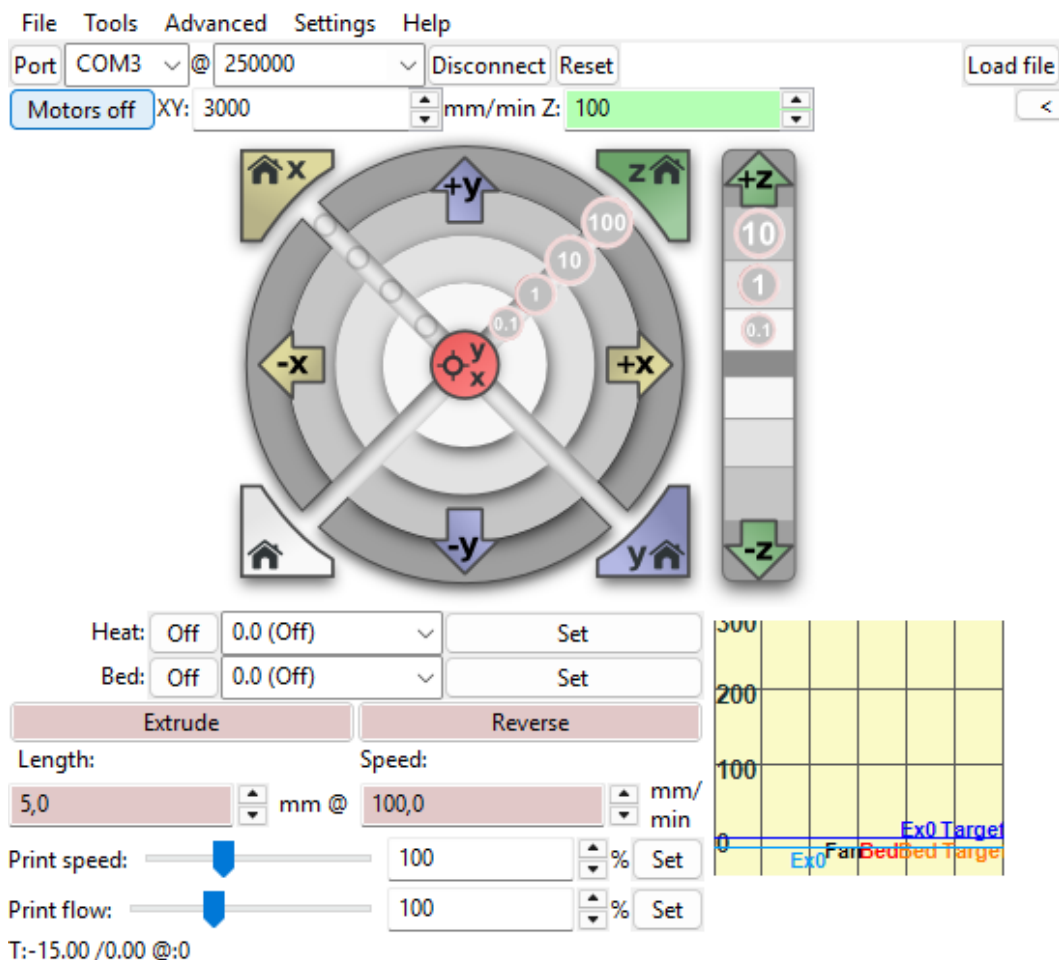


FIGURE 9 – Interface du logiciel permettant le déplacement manuel des axes X et Y

Cette méthode constitue une solution temporaire. Une étude complémentaire sera nécessaire afin d'identifier précisément l'origine du dysfonctionnement et de mettre en place une solution définitive.

29 Conclusion

Ce guide permet de comprendre le fonctionnement du programme STM32 utilisé pour le clavier braille autonome. Il explique la logique générale du code, le rôle des principales fonctions et la manière dont les boutons sont transformés en commandes d'impression braille.

Cette documentation peut être utilisée pour reprendre le projet, le modifier ou ajouter de nouvelles fonctionnalités.